SPARK ARCHITECTURE & ENGINE INTERNALS

Cluster Topology, JVM Memory Allocation, Shuffling, Catalyst & Resource Sizing

SPARK CORE | TUNGSTEN & PHOTON | MEMORY MODEL | CATALYST OPTIMIZER

Cluster Topology & Execution Hierarchy

Architecture

Component	Role & Responsibilities
Driver	Hosts `SparkSession`, DAGScheduler, TaskScheduler, Web UI (4040). Translates code to physical stages.
Cluster Mgr	Allocates physical compute resources (Databricks CM / YARN / K8s).
Worker Node	Physical VM hosting Executor JVM instances and NVMe shuffle disks.
Executor	JVM process running tasks in thread slots; holds execution/storage RAM.
Core / Slot	Thread processing slot. Optimal: **4-5 cores per executor**.

Execution Cascade: **Action** (collect/write) → **Job** → **Stages** (split by wide shuffles) → **Tasks** (1 task per partition per core).

Executor JVM Unified Memory Model

Memory

Segment	Size / Share	Purpose
Reserved	300 MB	Hardcoded for Spark internal engine structures
User Memory	40% (Heap - 300MB)	Custom data structures, UDF allocations, metadata
Unified Memory	60% (Heap - 300MB)	Dynamically shared between Execution & Storage
└ *Execution*	Dynamic pool	Shuffles, joins, sorts, aggregations (Can evict storage!)
└ *Storage*	Dynamic pool	Cached data (`df.cache()`); cannot evict execution!
Overhead	max(384MB, 10%)	Off-heap VM overhead, Py4J IPC, thread stacks

Off-Heap (Tungsten): `spark.memory.offHeap.enabled = true` allocates memory directly outside JVM, completely bypassing Garbage Collection pauses!

Transformations: Narrow vs Wide Dependencies

DAG Scheduler

Narrow (Pipelined)

- `map`, `filter`, `withColumn`, `union`
- Each parent partition used by at most 1 child partition.
- Zero network shuffle! Pipelined in memory within the **same stage**.

Wide (Shuffle Boundary)

- `groupBy`, `join`, `distinct`, `repartition`
- Multiple child partitions depend on parent partitions.
- Forces network exchange; terminates stage and starts new stage!

The Shuffle Engine: Write, Read & Spill

Shuffle

- **Shuffle Write:** Mappers write records to memory buffer; sorted and written to a single `.data` file with `.index` offset file on local NVMe disk.
- **Shuffle Read:** Reducers fetch their required blocks over Netty RPC from all mappers across the network into local memory buffers.
- **Spill (Memory) vs Spill (Disk):** Occurs when shuffle memory exceeds executor threshold. Data is serialized and flushed to disk (huge latency penalty!).

Triage Spills: If Spark UI shows Spill (Memory) / Spill (Disk), immediately increase executor RAM, enable AQE, or increase shuffle partitions!

Catalyst Optimizer & Tungsten Engine

Query Engine

1. Analysis
Resolves names against Unity Catalog.

2. Logical Opt
Predicate pushdown, column pruning.

3. Physical Plan
CBO picks optimal join (BHJ vs SMJ).

4. Tungsten CodeGen
Compiles loops into CPU L1/L2 registers.

UnsafeRow: Tungsten stores records in raw off-heap binary format without Java object wrapper overhead.

Cluster Sizing & Resource Allocation Math

Infrastructure

```
# Production Cluster Allocation Formula
# Example: 10 Nodes, 16 Cores & 64GB RAM each
1. Cores per Node = 16 - 1 (OS/YARN) = 15 Cores
2. Cores per Executor = 5 (Sweet spot for HDFS/ADLS IO throughput)
3. Executors per Node = 15 / 5 = 3 Executors
4. Total Executors = (10 Nodes * 3) - 1 (Driver) = 29 Executors
5. RAM per Node = 64GB - 1GB (OS) = 63GB
6. RAM per Executor = 63GB / 3 = 21GB
7. Memory Overhead = max(384MB, 0.10 * 21GB) = ~2.1GB
8. Executor Heap = 21GB - 2.1GB = ~18.9GB (Set: --executor-memory 19g)
```

Parallelism Rule: `spark.default.parallelism = total_executor_cores * 2 to 3` (For 145 cores = ~300-450 partitions).

⚙️ SPARK ARCHITECTURE & ENGINE INTERNALS

Photon Native Engine, AQE Dynamic Optimization, Spark UI Triage & Skew Mastery

AQE & SKEW | SPARK UI DEBUGGING | PHOTON C++

🚀 Databricks Photon Native Vectorized Engine

Photon

- **C++ Native Execution:** Replaces Tungsten Java bytecode execution path entirely with hand-optimized native C++.
- **SIMD Vectorization:** Processes batches of column values in CPU SIMD registers simultaneously (Single Instruction, Multiple Data).
- **Zero JVM Overhead:** Eliminates Java object allocation, GC pauses, and JIT deoptimizations for high-concurrency SQL and aggregations.

⚡ **Accelerated Operations:** Delta Parquet scanning, complex hash joins, high-cardinality aggregations, and string/regex operations execute up to **5x faster** on Photon.

🧠 Adaptive Query Execution (AQE) 3 Superpowers

Adaptive Tuning

Feature	Runtime Action	Config Knob
Dynamic Partition Coalescing	Merges tiny post-shuffle partitions to eliminate small-task scheduling overhead	`spark.sql.adaptive.coalescePartitions.enabled = true`
Dynamic Join Switching	Converts SortMergeJoin to BroadcastHashJoin if runtime stage output < 10MB	`spark.sql.adaptive.autoBroadcastJoinThreshold`
Dynamic Skew Join	Detects skewed partitions (>5x median & >256MB) and automatically splits them	`spark.sql.adaptive.skewJoin.enabled = true`

🔍 Spark UI Diagnostic & Triage Playbook

Diagnostics

Spark UI Tab	Metric / Symptom	Diagnosis & Fix
Stages Tab	Max task time >> 75th percentile	**Data Skew:** Single straggler task holding stage. Enable AQE skew join or salt keys.
Stages Tab	Spill (Memory) & Spill (Disk) > 0	**Memory Deficit:** Increase executor memory or raise `spark.sql.shuffle.partitions`.
Executors Tab	GC Time > 10% of Task Time	**Heap Exhaustion:** Large object allocations or oversized partitions. Enable Off-Heap memory.
SQL / Plan	SortMergeJoin instead of Broadcast	Check table statistics with `ANALYZE TABLE` or supply `/*+ BROADCAST */` hint.

⚠️ Data Skew Root Causes & Solutions

Skew Remediation

- **Root Causes:** High frequency keys (e.g. `NULL` customer IDs, default dummy values, power-law Pareto distributions).
- **Strategy 1 (AQE):** Ensure `spark.sql.adaptive.skewJoin.enabled = true` (Databricks default).
- **Strategy 2 (Null Separation):** Filter null keys into a separate stream; join only non-nulls; `UNION ALL` nulls back.
- **Strategy 3 (Broadcast):** If one table fits in RAM (<8GB on large clusters), broadcast it to eliminate shuffle completely!

💾 Storage Levels & Checkpoint Decision Matrix

Persistence

Storage Level	Space	CPU	RAM	Disk
`MEMORY_ONLY`	High	Lowest	Yes (Deser)	No
`MEMORY_ONLY_SER`	Low (3x less)	Low	Yes (Kryo)	No
`MEMORY_AND_DISK`	Medium	Med	Spills to Disk	Yes
`OFF_HEAP`	Low	Lowest	Tungsten RAM	No GC!

💡 **df.persist() vs df.checkpoint():** `persist()` keeps lineage for node recovery; `checkpoint()` writes to ADLS and **TRUNCATES lineage** graph (mandatory for iterative ML/Streaming).

⚙️ JVM Garbage Collection (G1GC) & Shuffle Math

Tuning Knobs

```
# Production G1GC Flags for Databricks / Spark
-XX:+UseG1GC
-XX:InitiatingHeapOccupancyPercent=35
-XX:G1ReservePercent=15
-XX:MaxGCPauseMillis=200

# Optimal Shuffle Partitions Calculation
# Rule: Target 100MB - 200MB per shuffle partition
Total Shuffle Input Data = 64 GB (= 65,536 MB)
Optimal Partitions = 65,536 MB / 128 MB = 512 Partitions
Set: spark.conf.set("spark.sql.shuffle.partitions", 512)
```

⚡ **AQE Advisory Size:** Set `spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728` (128MB) for automatic post-shuffle partition sizing.